Планировщик отпусков

КОРПОРАТИВНЫЙ ГРАФИК ОТПУСКОВ И СОГЛАСОВАНИЙ

Стенды, CI/CD и выпуск

Окружения, конвейеры, версии и эксплуатация · версия 0.3.0

28 октября 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Окружения 4

2.1 Локальное окружение 4

3 Конвейеры непрерывной интеграции 5

4 Ветки, версии и выпуск 7

5 Миграции базы данных 8

6 Резервное копирование и мониторинг 9

7 Доступы 10

1 О документе

Документ описывает окружения «Планировщика отпусков», конвейеры непрерывной интеграции в GitHub и Gitea, порядок выпуска версий, применение миграций, резервное копирование и мониторинг. Пошаговые инструкции для администратора стенда — «Руководство по развёртыванию».

ИСТОЧНИК	ЧТО ЗАДАЁТ
`docker-compose.yml`	Контейнеры стенда: postgres, server, web, backup
`.github/workflows/ci.yml`	Конвейер GitHub Actions
`.gitea/workflows/server-ci.yml`	Конвейер Gitea Actions с развёртыванием стенда
`Makefile`	Типовые команды разработчика
`docs/versions.md`, `CHANGELOG.md`	История версий, авторство, технический журнал

2 Окружения

ОКРУЖЕНИЕ	НАЗНАЧЕНИЕ	ДАННЫЕ	КТО РАБОТАЕТ
Локальное	Разработка и отладка	Демо-данные (`make seed`)	Разработчик
CI (GitHub, Gitea)	Проверка каждого push и PR	Тестовая БД `vacation_test` , демо-данные для e2e интерфейса	Автоматика
Стенд Gitea	Демонстрация и приёмка версии	Том `db-data` стенда	Команда, заказчик
Рабочий контур	Работа организации	Рабочая БД, резервные копии	Сотрудники

2.1 Локальное окружение

КОМАНДА	ДЕЙСТВИЕ
`make up`	PostgreSQL в Docker
`make seed`	Миграции и демо-данные (131 сотрудник, 14 отделов)
`make server-dev` / `make web-dev`	API на `:8080` , SPA на `:5173` с прокси `/api`
`make test`	Тесты сервера на БД `vacation_test`
`cd web && npm test` / `npm run e2e`	Модульные тесты клиента / e2e интерфейса (Playwright)
`make lint`	ESLint сервера и клиента
`make sbom`	SBOM CycloneDX и отчёт лицензий
`make docs-diagrams` / `make docs`	Схемы в PNG / комплект документов в PDF и DOCX
`make compose-full`	Полный стенд в контейнерах на `:80`

Секреты — только в `server/.env` (образец `.env.example`): `DATABASE_URL` , `JWT_SECRET` не короче 32 символов. В git их нет.

3 Конвейеры непрерывной интеграции

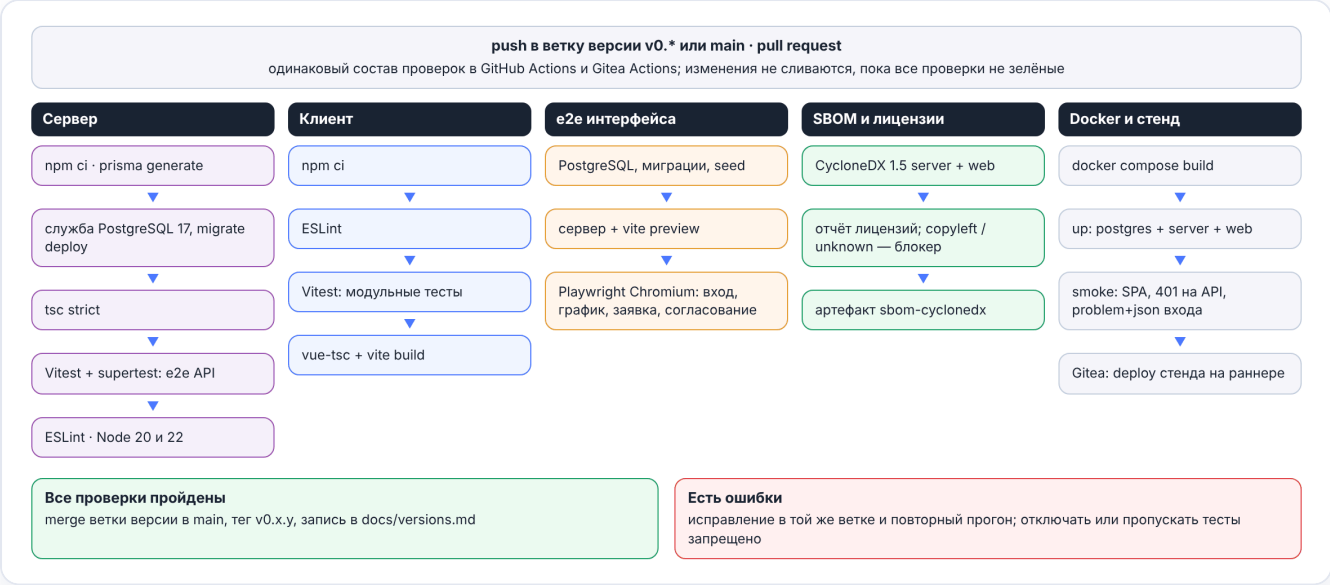


Рис. 1. Состав проверок CI

Оба конвейера запускаются на push в `main` и ветки `v0.*` (GitHub — также на pull request) и выполняют одинаковый набор проверок:

ЗАДАНИЕ	ШАГИ	ЧТО ЛОВИТ
Сервер	`npm ci`, `prisma generate`, миграции на чистой PostgreSQL 17, `tsc` strict, Vitest (модульные и e2e API), ESLint; Node 20 и 22	Ошибки логики, прав, транзакций, расхождение контракта и маршрутов, гонки
Клиент	`npm ci`, ESLint, Vitest модулей `lib/*`, `vue-tsc` + сборка Vite	Ошибки типов и вычислений дат, графика, статусов
e2e интерфейса	PostgreSQL, миграции, seed, сборки, Chromium; Playwright: вход, график, заявка с подбором дат и согласованием, команда, график работы, CSP	Регрессии экранов и сценариев, нарушения CSP
SBOM	CycloneDX 1.5 для сервера и клиента, `scripts/license-report.mjs`	Copyleft и неизвестные лицензии — блокер
Docker (GitHub)	Сборка образов и дымовой прогон полного стенда	Ошибки Dockerfile, nginx, старта с миграциями

ЗАДАНИЕ	ШАГИ	ЧТО ЛОВИТ
Deploy (Gitea)	После всех проверок: `docker compose -p vacation-planner up -d --build` на раннере `deploy` и дымовые пробы	Развёртывание стенда версии

Правила:

- красный конвейер блокирует слияние; отключать, пропускать или ослаблять тесты запрещено — исправляется причина;
- «нестабильный тест» не причина: в v0.3.0 так нашлась ошибка дымовой пробы (`CODE=$(curl …)` под `bash -e` падал при обрыве соединения до готовности API);
- дифф контракта, SBOM и документов — в том же PR, что и код.

4 Ветки, версии и выпуск



Рис. 2. Жизненный цикл выпуска

1. Номер версии `v0.x.y`: до первого релиза первая цифра 0; `x` растёт при крупных накопленных изменениях, `y` — при исправлениях.
2. Работа над версией — в ветке `v0.x.0`; отдельные истории — в ветках `story/<id>` от ветки версии с pull request по шаблону.
3. `main` получает только принятые версии: merge ветки версии и тег `v0.x.y` на merge-коммите.
4. Перед выпуском обновляются `docs/versions.md` (что получил пользователь, авторство), `CHANGELOG.md` (технически), версии в `package.json`, документы (`make docs`) и SBOM.
5. Мажорная версия (`v0.x.0`) выпускается только после полного прогона всех тестов в обоих конвейерах.

Порядок выпуска: зелёные конвейеры на ветке версии → обновлены история версий, журнал, документы, SBOM → merge в `main` с тегом → развёртывание стенда Gitea и дымовые пробы → приёмка → обновление рабочего контура (резервная копия → `docker compose up -d --build` → дымовые пробы).

5 Миграции базы данных

- Схема — `server/prisma/schema.prisma`; миграции применяются автоматически при старте контейнера `server` (`prisma migrate deploy`).
- Перед обновлением рабочего контура снимается внеочередная резервная копия.
- Опасные изменения (удаление, переименование) выполняются в два выпуска через добавляющие шаги.
- Seed идемпотентен и ничего не удаляет; на рабочем контуре его запуск не требуется.

6 Резервное копирование и мониторинг

ЗАДАЧА	РЕАЛИЗАЦИЯ
Резервное копирование	Сервис `backup`: `pg_dump -Fc` раз в `BACKUP_INTERVAL` секунд (сутки), хранение `BACKUP_KEEP_DAYS` (14) дней, том `db-backups`; запись атомарная (через `.tmp`)
Проверка восстановления	Раз в месяц — `pg_restore` последней копии в пустую БД и сверка числа сотрудников и заявок (руководство по развёртыванию)
Пробы	`/healthz` — процесс жив; `/readyz` — доступна БД; healthcheck PostgreSQL в compose
Журналы	stdout контейнеров; `docker compose logs`; ротация средствами Docker
Плановое (v0.5)	Метрики Prometheus (p95, ошибки, пул соединений), реплика PostgreSQL для RPO 15 минут

7 Доступы

РОЛЬ	ДОСТУП
Разработчик	Репозитории GitHub и Gitea; права на push в ветки историй
Владелец (ревью)	CODEOWNERS: `contracts/`, `docs/specs/`, миграции, `auth/`, workflows, зависимости
Оператор стенда	SSH к серверу, `.env` стенда вне git, раннер `deploy`
Администратор приложения	Учётная запись с признаком администратора внутри приложения